



Université de Technologie de Compiègne

TZ : Classification Naïve Bayésienne

Par Thomas Fallouh

Supervisé par M. Alaya et M. Bouzebda

Juillet 2024

Sommaire

Introduction	2
1 Rappel de probabilités	3
1.1 Propriétés des probabilités	3
1.1.1 Premier axiome	3
1.1.2 Deuxième axiome	3
1.1.3 Troisième axiome	4
1.1.4 Propriétés découlant des axiomes de Kolmogorov	4
1.2 Indépendance des événements	5
1.3 Conditionnement	5
2 Variables Aléatoires, Probabilités Conditionnelles et Vecteurs Aléatoires	8
2.1 Variables aléatoires	8
2.1.1 Définition	8
2.1.2 Exemples de Variables Aléatoires	8
2.1.3 Types de Variables Aléatoires	8
2.1.4 Variables Aléatoires Discrètes	8
2.1.5 Variables Aléatoires Continues	9
2.2 Fonctions de Caractérisation des Distributions : CDF, PDF et PMF	9
2.2.1 Fonction de Répartition Cumulative (CDF)	9
2.2.2 Fonction de Densité de Probabilité (PDF)	10
2.2.3 Fonction de Masse de Probabilité (PMF)	10
2.2.4 Exemple de CDF, PDF et PMF	10
2.2.5 Espérance et Variance	11
2.2.6 Lien entre Variables Aléatoires, Vecteurs aléatoires et Probabilités Conditionnelles	11
2.3 Vecteurs Aléatoires	12
2.3.1 Définition et Notions de Base	12
2.3.2 Fonction de Répartition	12
2.3.3 Indépendance des Vecteurs Aléatoires	12
2.3.4 Lois Conjointes et Marginales et leur Utilisation en Classification Naïve Bayésienne	12
2.3.5 Exemple avec des variables discrètes	13
2.4 Probabilités Conditionnelles	13
2.4.1 Définition et Exemple	13
2.4.2 Variables Aléatoires et Probabilités Conditionnelles	14
2.4.3 Application : Prédiction en Apprentissage Automatique	15
2.4.4 Indépendance	15
2.4.5 Théorème de Bayes	16
2.4.6 Applications et Exemples	16
2.4.7 Indépendance et Corrélation	17
3 Classification Bayésienne	18
3.1 Hypothèse naïve de Bayes	18
3.1.1 Présentation de l'hypothèse	18
3.1.2 Exemple d'application	19
3.2 Implémentation avec Python	21
3.2.1 Présentation du jeu de données	21
3.2.2 Programme Manuel	23
3.2.3 Construction du modèle	25
3.2.4 Programme avec Scikit-Learn	26
3.2.5 Output du programme	28
Conclusion	30

Introduction

Ce projet se consacre à l'exploration approfondie de la classification naïve bayésienne, une méthode de classification statistique basée sur le théorème de Bayes, qui est à la fois simple et puissante pour gérer les grands ensembles de données typiques des défis en intelligence artificielle. Nous débuterons notre étude par une exploration des concepts fondamentaux de la probabilité, incluant l'étude des probabilités conditionnelles ainsi que les variables aléatoires simples et vectorielles. Cette base théorique est essentielle pour comprendre et appliquer efficacement toute méthode statistique, en particulier dans le contexte de la modélisation probabiliste.

En progressant, nous nous pencherons sur le modèle de classification naïve bayésienne, en discutant son hypothèse centrale d'indépendance entre les prédicteurs ou variables. Cette hypothèse simplificatrice, bien que parfois critiquée, confère à la méthode sa facilité de mise en œuvre et contribue à son efficacité dans divers scénarios pratiques. Nous examinerons les implications de cette hypothèse à travers des exemples théoriques et des études de cas, ce qui permettra d'évaluer l'impact de l'indépendance supposée sur la précision du modèle.

La dernière étape consistera en l'implémentation pratique du modèle en Python. Nous développerons et testerons un modèle naïf bayésien sur des données réelles, utilisant des bibliothèques de programmation statistique modernes. Cette application concrète mettra en lumière comment les principes théoriques sont transformés en outils pratiques et efficaces pour la classification de données. À travers cette implémentation, nous évaluerons la performance du modèle, sa précision, afin de mieux comprendre ses capacités ainsi que ses limitations.

1 Rappel de probabilités

Vocabulaire des événements

Définitions :

- Une **expérience aléatoire** est une expérience dont les résultats possibles sont connus sans que l'on puisse déterminer lequel sera réalisé.
 - **Exemple** : lancer un dé, tirer une carte d'un jeu de 52 cartes.
- Un résultat possible d'une expérience aléatoire s'appelle une **issue**.
 - **Exemple** : obtenir un 3 en lançant un dé, tirer un As de cœur dans un Jeu de cartes.
- On appelle **événement** toute partie (ou sous-ensemble) de l'univers Ω qui correspond à l'ensemble de toutes les issues possibles d'une expérience aléatoire.
 - **Exemple** : obtenir un nombre pair en lançant un dé ($\{2, 4, 6\}$), tirer une figure (roi, dame, valet) d'un jeu de cartes.
- Toute partie de Ω ayant un seul élément est appelée **événement élémentaire**.
 - **Exemple** : obtenir un 5 en lançant un dé ($\{5\}$).
- $\emptyset$ est une partie de Ω , c'est un événement, appelé **événement impossible**.
- Ω est une partie de Ω , c'est un événement, appelé **événement certain**.

1.1 Propriétés des probabilités

La probabilité est une mesure qui quantifie la chance qu'un événement se produise. Elle respecte les axiomes de Kolmogorov.

Ils forment les bases fondamentales de la théorie des probabilités, établissant un cadre mathématique rigoureux pour l'analyse des événements aléatoires. Ces axiomes définissent la probabilité comme une mesure sur un espace probabilisé, garantissant la cohérence des calculs probabilistes. Voici les trois axiomes de Kolmogorov :

1.1.1 Premier axiome

Pour tout événement A , la probabilité $P(A)$ satisfait la condition suivante :

$$0 \leq P(A) \leq 1$$

C'est-à-dire que la probabilité d'un événement est représentée par un nombre réel compris entre 0 et 1. Cet axiome garantit que les probabilités sont toujours positives et ne dépassent jamais 1.

Exemple : Lorsque l'on lance un dé à six faces, la probabilité d'obtenir un nombre entre 1 et 6 est toujours non négative. Par exemple :

$$P(\{i\}) = \frac{1}{6} \quad \text{pour } i = 1, 2, 3, 4, 5, 6.$$

1.1.2 Deuxième axiome

La probabilité de l'univers (l'événement certain) est égale à 1 :

$$P(\Omega) = 1$$

Cela signifie que la probabilité que l'un des résultats possibles de l'univers Ω se produise est égale à 1. En d'autres termes, l'univers représente l'ensemble de tous les résultats possibles d'une expérience aléatoire, et la somme des probabilités de chaque résultat doit toujours être égale à 1, car un résultat dans Ω se réalisera forcément.

Exemple : Pour un dé non truqué à six faces, l'univers Ω est constitué des résultats possibles $\{1, 2, 3, 4, 5, 6\}$, la probabilité associée à chaque valeur vaut

$$\frac{1}{6}$$

. La somme des probabilités des issues doit donc être égale à 1 :

$$P(\{1, 2, 3, 4, 5, 6\}) = P(\{1\}) + P(\{2\}) + \cdots + P(\{6\}) = \frac{1}{6} + \frac{1}{6} + \cdots + \frac{1}{6} = 1$$

1.1.3 Troisième axiome

Pour toute suite d'événements $A_1, A_2, \dots$ deux à deux disjoints (c'est-à-dire incompatibles), la probabilité de la réunion de ces événements est égale à la somme des probabilités individuelles :

$$P\left(\bigcup_{i=1}^{+\infty} A_i\right) = \sum_{i=1}^{+\infty} P(A_i)$$

Cet axiome, connu sous le nom de additivité dénombrable, stipule que la probabilité d'un événement constitué de l'union dénombrable d'événements disjoints est égale à la somme des probabilités de ces événements.

Exemple : Si nous lançons un dé non truqué et que $A_1 = \{1\}$, $A_2 = \{2\}$, et ainsi de suite, alors les événements $A_1, A_2, \dots, A_6$ sont disjoints, et la probabilité de l'union de ces événements est la somme de leurs probabilités :

$$P(A_1 \cup A_2 \cup \cdots \cup A_6) = P(A_1) + P(A_2) + \cdots + P(A_6) = \frac{1}{6} + \frac{1}{6} + \cdots + \frac{1}{6} = 1$$

1.1.4 Propriétés découlant des axiomes de Kolmogorov

À partir des axiomes de Kolmogorov, plusieurs propriétés importantes peuvent être dérivées.

Probabilité de l'ensemble vide : La probabilité de l'ensemble vide est nulle.

$$P(\emptyset) = 0$$

Exemple : Il est impossible de tirer un nombre supérieur à 6 avec un dé à six faces. Ainsi, la probabilité de cet événement est 0.

Complémentarité : La probabilité du complément d'un événement A est déduite de la probabilité de A .

$$P(A^c) = 1 - P(A)$$

Exemple : Si A est l'événement "obtenir un nombre pair" en lançant un dé non truqué, alors $P(A) = \frac{1}{2}$. Le complément de A (obtenir un nombre impair) a donc une probabilité de $P(A^c) = \frac{1}{2}$.

Monotonie : Si A est un sous-ensemble de B , alors la probabilité de A est inférieure ou égale à la probabilité de B .

$$A \subseteq B \implies P(A) \leq P(B)$$

Exemple : Supposons que A est l'événement "obtenir un 2" en lançant un dé non truqué et B est l'événement "obtenir un nombre pair". Ici, $A \subseteq B$ et donc $P(A) \leq P(B)$, avec $P(A) = \frac{1}{6}$ et $P(B) = \frac{1}{2}$.

Union de deux événements quelconques : La probabilité de l'union de deux événements A et B est donnée par :

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Exemple : Si A est l'événement "obtenir un nombre pair" et B est l'événement "obtenir un nombre supérieur à 4", alors :

$$\begin{aligned} A &= \{2, 4, 6\}, & B &= \{5, 6\}, & A \cap B &= \{6\} \\ P(A \cup B) &= P(A) + P(B) - P(A \cap B) = \frac{2}{3} \end{aligned}$$

L'événement contraire de A , noté A^c , est l'ensemble de tous les événements élémentaires qui ne sont pas dans A .

$$P(A^c) = 1 - P(A)$$

Exemple : Si A est l'événement "obtenir un nombre pair" en lançant un dé non truqué, alors A^c est l'événement "obtenir un nombre impair".

1.2 Indépendance des événements

Deux événements A et B sont dits indépendants si la réalisation de l'un n'affecte pas la réalisation de l'autre. La probabilité que deux événements indépendants se réalisent dans une même expérience aléatoire est égale au produit de leurs probabilités. Ainsi, si A et B sont des événements d'un espace probabilisé U , on a l'égalité :

$$P(A \cap B) = P(A) \cdot P(B)$$

Exemple : Soit l'expérience A qui consiste à lancer un dé non truqué à six faces numérotées de 1 à 6, puis l'expérience B qui consiste à tirer ensuite une carte dans un jeu de 8 cartes (as et rois). Ces deux expériences n'ont pas de lien entre eux et la réalisation de l'un n'influence pas la réalisation de l'autre.

Par exemple, si l'événement A est "obtenir un 4 sur le dé" et l'événement B est "tirer un As", alors la probabilité conjointe $P(A \text{ et } B)$ serait la probabilité d'obtenir simultanément un 4 sur le dé et un As sur la carte.

La probabilité de A et B est :

$$P(A \cap B) = P(A) \cdot P(B) = \frac{1}{6} \times \frac{1}{8} = \frac{1}{48}$$

1.3 Conditionnement

Le conditionnement en probabilité est un concept fondamental qui permet de mettre à jour les probabilités d'événements ou de variables aléatoires en fonction de nouvelles informations ou de conditions spécifiques. Ce processus est crucial pour affiner les prédictions et les analyses, notamment dans des contextes où les informations évoluent au cours du temps.

Définition formelle: La probabilité conditionnelle de l'événement A sachant l'événement B est notée $P(A|B)$ et est définie par la formule :

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad \text{pour } P(B) > 0$$

Cette formule signifie que la probabilité que l'événement A se produise, sachant que B s'est produit, est égale à la probabilité de l'intersection des événements A et B divisée par la probabilité de B .

Exemple : Supposons un sac contenant 3 boules rouges et 2 boules bleues. Soit l'expérience une boule tirée au hasard:

- Soit A l'événement "la boule est rouge".
- Soit B l'événement "une boule est tirée".

La probabilité de A est :

$$P(A) = \frac{\text{nombre de boules rouges}}{\text{nombre total de boules}} = \frac{3}{5}$$

La probabilité conditionnelle de A sachant B est :

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(A)}{P(B)} = \frac{\frac{3}{5}}{1} = \frac{3}{5}$$

Puisque B est l'événement certain que "une boule est tirée", $P(B) = 1$.

Propriétés de la Probabilité Conditionnelle :

- **Multiplication des probabilités conditionnelles :** La probabilité que les événements A et B se produisent est :

$$P(A \cap B) = P(A|B) \cdot P(B)$$

- **Formule des probabilités totales :** Si $B_1, B_2, \dots, B_n$ forment une partition de l'espace échantillon, alors pour tout événement A :

$$P(A) = \sum_{i=1}^n P(A|B_i) \cdot P(B_i)$$

Exemple : Supposons un système de production avec deux machines M_1 et M_2 . La probabilité que M_1 fabrique un produit défectueux est de 0,1 et celle de M_2 est de 0,2. M_1 fabrique 60% des produits et M_2 40%.

- Soit D l'événement "produit défectueux".
- Soit M_1 et M_2 les événements indiquant quelle machine a fabriqué le produit.

La probabilité de D est :

$$P(D) = P(D|M_1)P(M_1) + P(D|M_2)P(M_2) = 0.1 \cdot 0.6 + 0.2 \cdot 0.4 = 0.06 + 0.08 = 0.14$$

En résumé, le conditionnement permet de calculer des probabilités dans des situations où nous avons une information supplémentaire sur la réalisation d'un autre événement.

Corollaire :

$$P(A \cap B) = P(A) \times P(B|A) = P(B) \times P(A|B)$$

Démonstration :

$$P(B|A) = \frac{P(A \cap B)}{P(A)} \implies P(A \cap B) = P(B|A) \times P(A)$$

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \implies P(A \cap B) = P(A|B) \times P(B)$$

D'où :

$$P(A \cap B) = P(A) \times P(B|A) = P(B) \times P(A|B)$$

Exemple :

Les élèves de terminale d'un lycée ont passé un test de certification en anglais.

- 80% ont réussi le test.
- Parmi ceux qui ont réussi le test, 95% n'ont jamais redoublé.
- Parmi ceux qui ont échoué, 2% n'ont jamais redoublé.

On considère les événements : T : " l'élève a réussi le test "

$\bar{R}$: " l'élève n'a jamais redoublé ".

Quelle est la probabilité de l'événement : " L'élève a réussi le test et n'a jamais redoublé " ?

L'énoncé nous donne les probabilités suivantes :

$$P(T) = 0.8, \quad P(\bar{R}|T) = 0.95, \quad P(\bar{R}|\bar{T}) = 0.02$$

On cherche la probabilité de l'événement $T \cap \bar{R}$:

$$P(T \cap \bar{R}) = P(T) \times P(\bar{R}|T) = 0.8 \times 0.95 = 0.76$$

Donc, la probabilité que l'élève ait réussi le test et n'ait jamais redoublé est de 0,76, soit 76%.

Représentation par un Arbre Pondéré :

- Sur chaque branche, on indique la probabilité d'obtenir l'issue correspondante.
- La somme des probabilités des branches de même origine doit toujours être égale à 1.
- La probabilité d'un chemin complet est égale au produit des probabilités inscrites sur chaque branche du chemin.

Voici l'illustration de la situation de l'exemple précédent à l'aide d'un arbre pondéré.

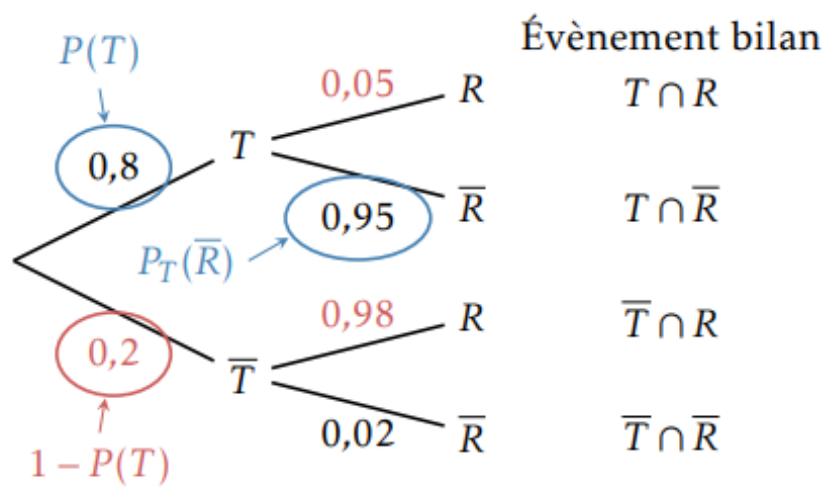


Figure 1: Arbre pondéré de l'exemple précédent

2 Variables Aléatoires, Probabilités Conditionnelles et Vecteurs Aléatoires

2.1 Variables aléatoires

2.1.1 Définition

Soit $(\Omega, \mathcal{A}, P)$ un espace probabilisé. On dit qu'une fonction $X : \Omega \rightarrow R$ est une variable aléatoire (réelle) si, pour tout intervalle I de R , $X^{-1}(I) \in \mathcal{A}$.

Un vecteur aléatoire est un m -uplet $(X_1, \dots, X_m)$ où chaque X_i est une variable aléatoire.

2.1.2 Exemples de Variables Aléatoires

Exemple 1 : Une urne contient une boule noire et une boule blanche. On tire avec remise deux boules de cette urne, et on note X le nombre de boules blanches obtenues. X est une variable aléatoire définie sur $\{B, N\}^2$ et qui vérifie :

$$X((B, B)) = 2, \quad X((B, N)) = X((N, B)) = 1, \quad X((N, N)) = 0$$

Exemple 2 : On lance un dé cubique, et on note X le nombre de lancers avant d'obtenir un 4. X est une variable aléatoire qui peut prendre toute valeur de N .

Exemple 3 : Soit A et B des points de coordonnées respectives $(1, 0)$ et $(1, 1)$ dans un repère orthonormé. On tire un point au hasard dans le triangle OAB , et on note X son abscisse. X est une variable aléatoire à valeurs dans $[0, 1]$.

Exemple 4 : Une puce se déplace aléatoirement dans un disque de centre 0 et de rayon 1. On note x et y ses abscisses et ordonnées. Alors (x, y) est un vecteur aléatoire réel.

2.1.3 Types de Variables Aléatoires

On distingue essentiellement trois types de variables aléatoires, suivant la nature de $X(\Omega)$, l'ensemble des valeurs prises par X :

1. $X(\Omega)$ est une partie finie de R . On dit que X est une variable aléatoire discrète finie (comme dans l'exemple 1 de la partie précédente).
2. $X(\Omega)$ est un sous-ensemble infini dénombrable de R . On dit que X est une variable aléatoire discrète infinie (comme dans l'exemple 2 de la partie précédente).
3. $X(\Omega)$ est une réunion finie ou dénombrable d'intervalles de R , et F . On dit que X est une variable aléatoire continue ou à densité. La fonction de répartition de X , peut s'écrire sous la forme :

$$F(X) = \int_{-\infty}^X f(t) dt$$

où f est une fonction positive ou nulle. On dit que X est une variable à densité (comme dans l'exemple 3 de la partie précédente).

2.1.4 Variables Aléatoires Discrètes

Une **variable aléatoire discrète** est une variable aléatoire qui ne peut prendre qu'un nombre fini ou dénombrable d'issues distinctes. Cela signifie que l'ensemble des valeurs possibles prises par une variable aléatoire discrète est soit fini, soit peut être mis en correspondance avec l'ensemble des entiers naturels N .

Exemple 1 : Une urne contient 3 boules rouges et 2 boules bleues. On tire une boule au hasard et on note X la couleur de la boule tirée. Calculons la probabilité d'obtenir une boule rouge.

La probabilité d'obtenir une boule rouge est :

$$P(X = \text{rouge}) = \frac{3}{5}$$

Exemple 2 : On lance un dé équilibré à six faces. Soit X la variable aléatoire représentant le résultat du lancer. Calculons la probabilité d'obtenir un nombre pair.

La probabilité d'obtenir un nombre pair (2, 4, ou 6) est :

$$P(X \in \{2, 4, 6\}) = \frac{3}{6} = \frac{1}{2}$$

2.1.5 Variables Aléatoires Continues

Une **variable aléatoire continue** est une variable aléatoire qui peut prendre n'importe quelle valeur dans un intervalle (fini ou infini) de l'ensemble des réels R . Contrairement à une variable aléatoire discrète, qui prend des valeurs distinctes et isolées, une variable aléatoire continue peut prendre une infinité de valeurs possibles dans un intervalle donné.

Exemple 1 : Soit X une variable aléatoire représentant la durée de vie (en années) d'un composant électronique, qui suit une loi exponentielle de paramètre $\lambda = 0.5$. Calculons la probabilité que le composant ait une durée de vie supérieure à 2 ans.

La probabilité recherchée est :

$$P(X > 2) = \int_2^{\infty} 0.5e^{-0.5x} dx = e^{-0.5 \times 2} = e^{-1} \approx 0.3679$$

Exemple 2 : Soit X une variable aléatoire uniformément distribuée sur l'intervalle $[0, 1]$. Calculons la probabilité que X soit inférieur à 0.3.

La probabilité recherchée est :

$$P(X < 0.3) = \int_0^{0.3} 1 dx = 0.3$$

2.2 Fonctions de Caractérisation des Distributions : CDF, PDF et PMF

Les fonctions de répartition cumulative (CDF : Cumulative Distribution Function), les fonctions de densité de probabilité (PDF : Probability Density Function) et les fonctions de masse de probabilité (PMF : Probability Mass Function) sont des outils essentiels dans la théorie des probabilités et les statistiques. Elles sont utilisées pour caractériser et analyser les distributions des variables aléatoires. Voici à quoi elles servent :

2.2.1 Fonction de Répartition Cumulative (CDF)

La CDF, notée $F_X(x)$, est définie pour une variable aléatoire X comme suit :

$$F_X(x) = P(X \leq x)$$

La CDF donne la probabilité que la variable aléatoire X prenne une valeur inférieure ou égale à x .

- **Calcul de Probabilités :** La CDF permet de calculer la probabilité que X se situe dans un certain intervalle $[a, b]$:

$$P(a \leq X \leq b) = F_X(b) - F_X(a)$$

- **Identification des Quartiles et Percentiles :** La CDF peut être utilisée pour trouver les quartiles, percentiles et autres mesures de position dans une distribution.
- **Détermination de la Médiane :** La médiane est la valeur m telle que $F_X(m) = 0.5$.
- **Comparaison des Distributions :** La CDF permet de comparer différentes distributions en observant leurs fonctions de répartition.

2.2.2 Fonction de Densité de Probabilité (PDF)

La PDF, notée $f_X(x)$, est définie pour une variable aléatoire continue X comme la dérivée de la CDF :

$$f_X(x) = \frac{d}{dx} F_X(x)$$

La PDF donne la densité de probabilité en un point donné. Contrairement à la CDF, la PDF n'est pas une probabilité, mais elle est proportionnelle à la probabilité.

- **Calcul de Probabilités** : Pour une variable aléatoire continue, la probabilité que X prenne une valeur dans un intervalle $[a, b]$ est donnée par l'intégrale de la PDF sur cet intervalle :

$$P(a \leq X \leq b) = \int_a^b f_X(x) dx$$

- **Détermination des Propriétés de la Distribution** : La PDF permet de caractériser la forme de la distribution, telle que la symétrie, l'asymétrie, etc.
- **Identification des Modes** : Les points où la PDF atteint des maxima locaux sont les modes de la distribution. Le mode correspond à la ou les valeurs les plus probables de la variable aléatoire. Dans une distribution unimodale, il y a un seul mode. En revanche, dans une distribution multimodale, plusieurs modes existent, chacun correspondant à un maximum local de la densité de probabilité.
- **Estimation de Paramètres** : La PDF est utilisée dans des techniques d'estimation statistique, comme le maximum de vraisemblance, pour estimer les paramètres de la distribution.

2.2.3 Fonction de Masse de Probabilité (PMF)

La PMF, notée $p_X(x)$, est définie pour une variable aléatoire discrète X comme suit :

$$p_X(x) = P(X = x)$$

La PMF donne la probabilité que la variable aléatoire X prenne la valeur x .

- **Calcul de Probabilités** : Pour une variable aléatoire discrète, la probabilité que X prenne une valeur dans un ensemble fini S est donnée par la somme de la PMF sur cet ensemble :

$$P(X \in S) = \sum_{x \in S} p_X(x)$$

- **Détermination des Propriétés de la Distribution** : La PMF permet de caractériser les valeurs spécifiques prises par une variable aléatoire discrète et leurs probabilités associées.
- **Identification des Modes** : Les points où la PMF atteint des maxima sont les modes de la distribution.
- **Estimation de Paramètres** : La PMF est utilisée pour estimer les paramètres (de position, de dispersion, de forme) de la distribution d'une variable aléatoire discrète .

2.2.4 Exemple de CDF, PDF et PMF

Exemple 1 : Loi Exponentielle

Variable aléatoire continue : La loi exponentielle modélise des événements continus, comme le temps entre les appels téléphoniques.

La CDF cumule les probabilités jusqu'à une valeur donnée de x . Pour la loi exponentielle, elle est définie comme suit :

$$F_X(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 - e^{-\lambda x} & \text{si } x \geq 0 \end{cases}$$

La PDF est utilisée pour des variables aléatoires continues, elle décrit la densité de probabilité à un point donné. Pour la loi exponentielle, la PDF est :

$$f_X(x) = \begin{cases} 0 & \text{si } x < 0 \\ \lambda e^{-\lambda x} & \text{si } x \geq 0 \end{cases}$$

Explication : La loi exponentielle est utilisée pour modéliser le temps entre des événements qui se produisent de manière continue et indépendamment à un taux constant. Par exemple, le temps entre les appels téléphoniques entrants à un standard téléphonique. Supposons que le temps entre les appels téléphoniques soit exponentiellement distribué avec un taux de $\lambda = 2$ appels par minute. La probabilité qu'il faille attendre plus de 2 minutes pour le prochain appel est donnée par :

$$P(X > 2) = \int_2^{\infty} 2e^{-2x} dx = e^{-2 \cdot 2} = e^{-4} \approx 0.0183$$

Exemple 2 : Loi Binomiale

Variable aléatoire discrète : La loi binomiale modélise un nombre discret de succès dans une série d'essais indépendants (par exemple, un nombre de faces obtenu lors de plusieurs lancers de pièce).

La PMF est utilisée pour les variables aléatoires discrètes, elle donne la probabilité exacte que la variable prenne une certaine valeur.

Pour une variable aléatoire discrète X suivant une loi binomiale $B(n, p)$, la PMF est :

$$p_X(x) = \binom{n}{x} p^x (1-p)^{n-x} \quad \text{pour } x = 0, 1, 2, \dots, n$$

Explication : La loi binomiale est utilisée pour modéliser le nombre de succès dans une séquence de n essais indépendants, chacun ayant deux issues possibles (succès ou échec) avec une probabilité constante p de succès. Par exemple, considérons une pièce de monnaie équilibrée (probabilité $p = 0.5$ de succès pour chaque lancer). Si nous lançons la pièce 10 fois, la probabilité d'obtenir exactement 5 faces est donnée par :

$$P(X = 5) = \binom{10}{5} (0.5)^5 (0.5)^{10-5} = \binom{10}{5} (0.5)^{10} = 252 \cdot (0.5)^{10} \approx 0.246$$

2.2.5 Espérance et Variance

Espérance : Elle permet d'estimer la tendance centrale des données pour chaque caractéristique conditionnée à chaque classe. Dans un modèle gaussien, cette espérance est utilisée pour centrer la distribution normale qui modélise chaque caractéristique.

Variance : Elle mesure l'étalement des données autour de l'espérance. Dans le classificateur Naïve Bayes, la variance est utilisée pour définir la largeur de la distribution gaussienne. Une petite variance signifie que les valeurs sont très concentrées autour de la moyenne, tandis qu'une grande variance indique que les valeurs sont plus dispersées.

Espérance mathématique (moyenne) :

$$E(X) = \sum_i x_i P(X = x_i) \quad (\text{discrète})$$

$$E(X) = \int_{-\infty}^{\infty} x f_X(x) dx \quad (\text{continue})$$

Variance et écart-type :

$$\begin{aligned} \text{Var}(X) &= E[(X - E(X))^2] \\ \sigma_X &= \sqrt{\text{Var}(X)} \end{aligned}$$

2.2.6 Lien entre Variables Aléatoires, Vecteurs aléatoires et Probabilités Conditionnelles

Dans de nombreux problèmes de classification, comme celui que nous rencontrons dans l'algorithme Naïve Bayes, il ne suffit pas d'examiner des variables aléatoires isolées. Il devient crucial de considérer des vecteurs aléatoires, composés de plusieurs caractéristiques (ou variables aléatoires) qui décrivent une observation. Ces vecteurs aléatoires permettent de modéliser simultanément plusieurs attributs qui influencent la classe à prédire. Le classificateur Naïve Bayes repose aussi sur les probabilités conditionnelles, qui permettent de calculer la probabilité d'une classe donnée en fonction des valeurs observées de ces caractéristiques.

2.3 Vecteurs Aléatoires

2.3.1 Définition et Notions de Base

Un vecteur aléatoire, également connu sous le nom de variable aléatoire multidimensionnelle, est une extension d'une variable aléatoire à plusieurs dimensions. Alors qu'une variable aléatoire réelle associe à chaque éventualité un nombre réel, un vecteur aléatoire associe à chaque éventualité un vecteur de R^n .

Un vecteur aléatoire $X = (X_1, X_2, \dots, X_n)$ est défini sur un espace de probabilités $(\Omega, \mathcal{F})$ et prend des valeurs dans R^n . Les fonctions $X_1, X_2, \dots, X_n$ sont des variables aléatoires réelles appelées composantes du vecteur aléatoire X .

Pour que X soit considéré comme un vecteur aléatoire, il doit être mesurable. Cela signifie que pour tout ensemble mesurable B de R^n , l'ensemble $X^{-1}(B) \in \mathcal{F}$.

2.3.2 Fonction de Répartition

La fonction de répartition d'un vecteur aléatoire permet de calculer la probabilité que chacune de ses composantes prenne une valeur inférieure ou égale à un certain seuil. Lorsque les composantes du vecteur aléatoire sont indépendantes, la probabilité conjointe de plusieurs événements se factorise en produit des probabilités individuelles.

La fonction de répartition d'un vecteur aléatoire $X = (X_1, \dots, X_n)$ est définie comme suit :

$$F(x_1, \dots, x_n) = P(X_1 \leq x_1, \dots, X_n \leq x_n)$$

2.3.3 Indépendance des Vecteurs Aléatoires

Deux vecteurs aléatoires X et Y sont indépendants si la probabilité que ces vecteurs prennent une certaine valeur est égale au produit des probabilités que chaque vecteur prenne cette valeur individuellement :

$$P(X = x \text{ et } Y = y) = P(X = x) \cdot P(Y = y)$$

2.3.4 Lois Conjointes et Marginales et leur Utilisation en Classification Naïve Bayésienne

Les lois conjointes et marginales jouent un rôle crucial en statistique et en apprentissage automatique, particulièrement dans des approches telles que la classification naïve bayésienne.

Loi conjointe : La loi conjointe décrit la probabilité que plusieurs variables aléatoires (ou vecteurs aléatoires) prennent simultanément certaines valeurs. Par exemple, considérons un vecteur aléatoire (X, Y) prenant des valeurs dans R^2 . La loi conjointe de (X, Y) est donnée par la probabilité que X et Y se trouvent dans des intervalles spécifiques I et J :

$$P((X, Y) \in I \times J) = P(X \in I \text{ et } Y \in J).$$

Lois marginales : Les lois marginales sont obtenues en sommant ou en intégrant la loi conjointe sur toutes les valeurs possibles de l'autre variable. Cela permet d'analyser chaque variable individuellement. Par exemple, la loi marginale de X est obtenue en intégrant la loi conjointe sur toutes les valeurs possibles de Y :

$$P(X \in I) = \int_J P((X, Y) \in I \times J) dy.$$

Lien avec Naïve Bayes

Dans la classification Naïve Bayes, nous considérons un vecteur de caractéristiques $\mathbf{X} = (X_1, X_2, \dots, X_n)$ représentant plusieurs variables aléatoires correspondant à des attributs d'une observation. L'hypothèse clé du modèle est que, conditionnellement à la classe C , les caractéristiques $X_1, X_2, \dots, X_n$ sont indépendantes les unes des autres. Ainsi, la loi conjointe des caractéristiques se factorise en un produit de lois marginales, facilitant ainsi les calculs de probabilités conditionnelles nécessaires pour la classification. :

$$P(\mathbf{X}|C) = P(X_1|C) \cdot P(X_2|C) \cdot \dots \cdot P(X_n|C).$$

Cela simplifie le calcul des probabilités conditionnelles, car il suffit de multiplier les probabilités de chaque caractéristique prise individuellement.

2.3.5 Exemple avec des variables discrètes

Prenons l'exemple où deux nombres entiers sont tirés au hasard dans $\{-1, 1\}$, avec X représentant leur somme et Y leur produit. L'univers des tirages est $\{-1, 1\}^2$, et les valeurs possibles pour X et Y sont respectivement $\{-2, 0, 2\}$ et $\{-1, 1\}$. Voici les probabilités conjointes obtenues:

$$\begin{aligned} P(X = 2, Y = 1) &= \frac{1}{4}, & (\text{cas où l'on tire } 1 \text{ et } 1), \\ P(X = 2, Y = -1) &= 0, \\ P(X = 0, Y = 1) &= 0, \\ P(X = 0, Y = -1) &= \frac{1}{2}, \\ P(X = -2, Y = 1) &= \frac{1}{4}, \\ P(X = -2, Y = -1) &= 0. \end{aligned}$$

Le tableau suivant résume les probabilités conjointes pour chaque couple (X, Y) , ainsi que les probabilités marginales pour chaque X et Y :

$X \backslash Y$	-1	1	$P(X = x_i)$
2	0	1/4	1/4
0	1/2	0	1/2
-2	0	1/4	1/4
$P(Y = y_j)$	1/2	1/2	1

Table 1: Loi conjointe de X et Y et Loi marginale

Pour obtenir les lois marginales à partir de ce tableau, on procède comme suit:

$$P(X = x_i) = \sum_{j=1}^q P(X = x_i, Y = y_j), \quad P(Y = y_j) = \sum_{i=1}^p P(X = x_i, Y = y_j).$$

Cela illustre comment les informations contenues dans la loi conjointe peuvent être utilisées pour récupérer les lois marginales.

2.4 Probabilités Conditionnelles

2.4.1 Définition et Exemple

La probabilité conditionnelle est une mesure de la probabilité qu'un événement A se produise, sachant qu'un autre événement B s'est déjà produit. Elle est notée $P(A|B)$ et est définie par la formule suivante :

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad \text{pour } P(B) > 0$$

Cette définition exprime la probabilité de A en restreignant l'univers des possibles à B .

Exemple 1 : Lancer de Dé

Considérons un dé à six faces. Soit A l'événement "obtenir un nombre pair" et B l'événement "obtenir un nombre supérieur à 3".

- $A = \{2, 4, 6\}$
- $B = \{4, 5, 6\}$

La probabilité de A sachant B est :

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(\{4, 6\})}{P(\{4, 5, 6\})} = \frac{2/6}{3/6} = \frac{2}{3}$$

2.4.2 Variables Aléatoires et Probabilités Conditionnelles

Le conditionnement s'applique également aux variables aléatoires. Soit X et Y deux variables aléatoires définies sur le même espace probabilisé. La distribution conditionnelle de X sachant $Y = y$ est notée $f_{X|Y}(x|y)$ et se définit comme :

$$f_{X|Y}(x|y) = \frac{f_{X,Y}(x,y)}{f_Y(y)}, \quad \text{pour } f_Y(y) > 0$$

où $f_{X,Y}(x,y)$ est la densité jointe de X et Y , et $f_Y(y)$ est la densité marginale de Y .

Exemple : Distribution Normale Conjointe

Considérons deux variables aléatoires X et Y qui suivent une distribution normale conjointe. Nous allons voir comment calculer la densité conditionnelle de X sachant $Y = y$ en utilisant les densités jointe et marginale.

Contexte : Supposons que X représente la taille d'une personne (en cm) et Y représente son poids (en kg). Ces deux variables aléatoires sont corrélées et suivent une distribution normale conjointe. Voici les paramètres :

- La taille X suit une distribution normale avec une moyenne de 170 cm et une variance de 9 cm².
- Le poids Y suit une distribution normale avec une moyenne de 70 kg et une variance de 16 kg².
- La corrélation entre X et Y est donnée par $\rho = 0.8$.

Le vecteur aléatoire $\begin{pmatrix} X \\ Y \end{pmatrix}$ suit donc une distribution normale conjointe :

$$\begin{pmatrix} X \\ Y \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} 170 \\ 70 \end{pmatrix}, \begin{pmatrix} 9 & 0.8 \cdot 3 \cdot 4 \\ 0.8 \cdot 3 \cdot 4 & 16 \end{pmatrix} \right)$$

La diagonale de la matrice contient les variances des deux variables : 9 pour la taille X et 16 pour le poids Y .

Les termes hors diagonale $0.8 \cdot 3 \cdot 4$ correspondent à la covariance entre X et Y , qui est calculée à partir de la corrélation $\rho = 0.8$, des écarts types de X (3 cm) et de Y (4 kg). La covariance est donnée par la formule

$$\text{Cov}(X, Y) = \rho \cdot \sigma_X \cdot \sigma_Y,$$

où σ_X et σ_Y sont les écarts types de X et Y .

Ainsi, la covariance est

$$0.8 \cdot 3 \cdot 4 = 9.6,$$

donc la matrice de covariance devient :

$$\Sigma = \begin{pmatrix} 9 & 9.6 \\ 9.6 & 16 \end{pmatrix}.$$

Puisque le coefficient de corrélation $\rho = 0.8$ est proche de 1, cela indique une forte relation positive entre la taille et le poids, c'est-à-dire que lorsque la taille augmente, le poids a tendance à augmenter de manière proportionnelle. Cette matrice de covariance décrit la relation entre la taille et le poids, et la corrélation ρ indique qu'il existe une forte relation entre ces deux variables.

Densités Jointe et Marginale

- **Densité jointe** $f_{X,Y}(x,y)$: La densité jointe de X et Y décrit la probabilité que la taille soit égale à x et que le poids soit égal à y simultanément. Dans le cas de notre exemple, cela correspond à une distribution normale conjointe bidimensionnelle.
- **Densité marginale** $f_Y(y)$: La densité marginale de Y est obtenue en intégrant la densité jointe sur toutes les valeurs possibles de X . Cela représente la distribution de la variable Y (poids) indépendamment de X (taille). Dans notre exemple, cette densité marginale est simplement une distribution normale pour le poids avec une moyenne de 70 kg et une variance de 16 kg².

Calcul de la Densité Conditionnelle La densité conditionnelle de X sachant que $Y = y$ est donnée par :

$$f_{X|Y}(x|y) = \frac{f_{X,Y}(x,y)}{f_Y(y)}, \quad \text{pour } f_Y(y) > 0$$

Dans le cas d'une distribution normale conjointe, cette densité conditionnelle suit également une distribution normale avec des paramètres ajustés.

Distribution Conditionnelle Nous souhaitons calculer la distribution conditionnelle de la variable X (la taille), sachant que $Y = 75$ kg (le poids). Cette distribution est une loi normale avec une nouvelle moyenne conditionnelle donnée par :

$$\mu_{X|Y} = \mu_X + \rho \cdot \frac{\sigma_X}{\sigma_Y} \cdot (Y - \mu_Y)$$

Avec :

- $\mu_X = 170$ cm : moyenne initiale de la taille,
- $\rho = 0,8$: coefficient de corrélation entre la taille et le poids,
- $\sigma_X = 3$ cm : écart type de la taille,
- $\sigma_Y = 4$ kg : écart type du poids,
- $Y = 75$ kg : valeur observée du poids,
- $\mu_Y = 70$ kg : moyenne du poids.

Substituons les valeurs dans la formule :

$$\mu_{X|Y} = 170 + 0,8 \cdot \frac{3}{4} \cdot (75 - 70) = 170 + 0,6 \cdot 5 = 170 + 3 = 173 \text{ cm}$$

- Une variance conditionnelle donnée par :

$$\sigma_{X|Y}^2 = (1 - 0,8^2) \cdot 9 = 0,36 \cdot 9 = 3,24 \text{ cm}^2$$

Ainsi, la distribution conditionnelle de la taille X , sachant que le poids $Y = 75$ kg, est une loi normale :

$$X|(Y = 75) \sim \mathcal{N}(173, 3.24)$$

Cela signifie que, sachant qu'une personne pèse 75 kg, sa taille suit une distribution normale avec une moyenne de 173 cm et un écart-type d'environ 1.8 cm.

Grâce à cette distribution conditionnelle, nous pouvons estimer que, si une personne pèse 75 kg, sa taille sera probablement centrée autour de 173 cm, avec une incertitude représentée par une variance de 3.24 cm².

2.4.3 Application : Prédiction en Apprentissage Automatique

Le conditionnement est largement utilisé en apprentissage automatique, notamment dans les modèles de classification et de régression. Par exemple, dans un modèle de régression linéaire, la prédiction de la variable cible Y basée sur les caractéristiques X repose sur le conditionnement de Y par X .

En résumé, le conditionnement permet de mettre à jour les probabilités et les distributions en tenant compte des informations disponibles. Il s'agit d'un outil puissant pour affiner les analyses probabilistes et statistiques, offrant une base solide pour la prise de décision et les prédictions dans des contextes variés.

2.4.4 Indépendance

Définition : Les événements A et B sont dits indépendants lorsque $P(A \cap B) = P(A) \times P(B)$.

Propriété : A et B sont indépendants si et seulement si $P(B|A) = P(B)$.

Démonstration :

$$P(A \cap B) = P(A) \times P(B|A)$$

Ainsi :

$$P(A \cap B) = P(A) \times P(B) \iff P(A) \times P(B|A) = P(A) \times P(B) \iff P(B|A) = P(B)$$

Remarque : L'indépendance de A et B signifie donc que la réalisation de l'un n'impacte pas la probabilité que l'autre se réalise.

Propriété : Si A et B sont indépendants, alors B et A sont également indépendants.

Exemple : Soient A et B deux événements tels que $P(A) = 0.8$, $P(B) = 0.35$ et $P(A \cap B) = 0.28$. Montrons que A et B sont indépendants.

$$P(A) \times P(B) = 0.8 \times 0.35 = 0.28 = P(A \cap B)$$

Donc A et B sont indépendants.

NB : On le retrouve aussi en calculant en amont $P(B|A)$ ou $P(A|B)$.

$$P(B|A) = \frac{P(A \cap B)}{P(A)} = \frac{0.28}{0.8} = 0.35 = P(B)$$

2.4.5 Théorème de Bayes

$$P(B|A) = \frac{P(A|B) \cdot P(B)}{P(A)}, \quad \text{pour } P(A) > 0$$

Chaque terme du théorème de Bayes a une dénomination usuelle. Le théorème de Bayes établit une relation fondamentale entre différentes probabilités, permettant de mettre à jour nos croyances initiales basées sur de nouvelles informations.

Selon ce théorème, la probabilité **a posteriori** de B étant donné A , notée $P(B|A)$, se calcule en multipliant la probabilité **a priori** de B , notée $P(B)$, par la fonction de vraisemblance de A par rapport à B , soit $P(A|B)$, et en divisant le résultat par la probabilité marginale de A , $P(A)$.

La probabilité **a priori** $P(B)$ représente notre estimation de la probabilité de B avant de prendre en compte l'événement A . Elle sert de base dans le calcul et est ajustée une fois que l'événement A est observé.

D'autre part, $P(B|A)$ reflète la probabilité révisée de B après avoir pris en compte A , intégrant ainsi l'influence directe de l'observation de A sur notre évaluation de B . Cette probabilité est particulièrement importante car elle nous permet d'ajuster nos croyances à la lumière de nouvelles données.

Quant à $P(A|B)$, il représente la probabilité d'observer A si B est vrai, ce qu'on appelle la vraisemblance. Cette valeur est essentielle pour mettre à jour la probabilité de B en $P(B|A)$, car elle montre à quel point les données observées A sont cohérentes avec l'hypothèse B . Enfin, $P(A)$ joue un rôle de normalisation, garantissant que la somme des probabilités mises à jour pour toutes les hypothèses B possibles soit égale à 1.

En combinant ces éléments, le théorème de Bayes nous fournit un moyen puissant de réviser les probabilités initiales en fonction des preuves disponibles.

2.4.6 Applications et Exemples

Exemple : Supposons un système de production avec deux machines M_1 et M_2 . La probabilité que M_1 produise un produit défectueux est de 0.1 et celle de M_2 est de 0.2. M_1 produit 60% des produits et M_2 40%.

Soit D l'événement "produit défectueux". Soit M_1 et M_2 les événements indiquant quelle machine a produit le produit.

La probabilité de D est :

$$P(D) = P(D|M_1)P(M_1) + P(D|M_2)P(M_2) = 0.1 \cdot 0.6 + 0.2 \cdot 0.4 = 0.06 + 0.08 = 0.14$$

Si l'on souhaite savoir quelle est la probabilité que le produit défectueux provienne de la machine M_1 , on peut utiliser la règle de Bayes :

$$P(M_1|D) = \frac{P(D|M_1) \cdot P(M_1)}{P(D)} = \frac{0.1 \cdot 0.6}{0.14} = \frac{0.06}{0.14} \approx 0.4286$$

2.4.7 Indépendance et Corrélation

Deux variables aléatoires X et Y sont indépendantes si :

$$P(X \leq x, Y \leq y) = P(X \leq x) \cdot P(Y \leq y) \quad \text{pour tous } x, y$$

Exemples de Vecteurs Aléatoires :

- Analyse des notes de plusieurs examens d'un étudiant.
- Mesures de différentes caractéristiques physiques d'un groupe de personnes.

Applications en Statistiques Multivariées :

- Analyse en composantes principales (ACP) pour réduire la dimensionnalité des données.

3 Classification Bayésienne

3.1 Hypothèse naïve de Bayes

3.1.1 Présentation de l'hypothèse

Après avoir examiné la probabilité conditionnelle et la règle de Bayes, nous abordons le concept de Naïve Bayes, une approche simplifiée mais puissante pour prédire des résultats à partir de multiples indices ou variables prédictives. Originellement, cette méthode était appliquée avec une seule variable pour faciliter la compréhension, mais en réalité, Naïve Bayes est souvent utilisé avec de multiples indices qui rendent les calculs plus complexes. Pour gérer cette complexité, Naïve Bayes fait une hypothèse fondamentale : il considère chaque indice comme indépendant des autres. Cette hypothèse d'indépendance, bien que souvent fautive dans les scénarios réels, simplifie considérablement les calculs et constitue la raison pour laquelle l'approche est qualifiée de "naïve".

Le théorème de Bayes est formulé comme suit:

$$P(C | X) = \frac{P(X | C) \times P(C)}{P(X)}$$

où:

- $P(C | X)$ est la probabilité a posteriori de la classe C étant donné l'observation X .
- $P(X | C)$ est la probabilité de l'observation X étant donné la classe C , aussi appelée vraisemblance.
- $P(C)$ est la probabilité a priori de la classe C .
- $P(X)$ est la probabilité totale de l'observation X .

Probabilités A Priori : Avant de voir les données, on estime la probabilité de chaque classe, $P(C)$, souvent calculée comme la fréquence relative de chaque classe dans le jeu de données d'entraînement.

Vraisemblance (Probabilité Conditionnelle) : Pour chaque classe, on calcule la vraisemblance des données observées. Sous l'hypothèse d'indépendance (qui est la "naïveté" de Naïve Bayes), la vraisemblance d'un ensemble de caractéristiques $X = \{x_1, x_2, \dots, x_n\}$, est le produit des vraisemblances individuelles de chaque caractéristique :

$$P(X | C) = \prod_{i=1}^n P(x_i | C)$$

où $P(x_i | C)$ est la probabilité de la caractéristique x_i étant donné la classe C .

Probabilité Marginale $P(X)$: Cette probabilité normalise la probabilité a posteriori pour s'assurer qu'elle somme à 1 sur toutes les classes possibles, mais elle n'est souvent pas calculée directement car elle est constante pour toutes les classes.

Classification d'une Nouvelle Instance : Naïve Bayes sélectionne la classe qui maximise la probabilité a posteriori, simplifiée à:

$$\hat{C} = \arg \max_C (P(X | C) \times P(C))$$

Formule et Interprétation du Naïve Bayes

La formule de Naïve Bayes se présente plus intuitivement donc comme suit :

$$P(\text{Résultat} | \text{Indices Multiples}) = \frac{P(\text{Indice}_1 | \text{Résultat}) \times P(\text{Indice}_2 | \text{Résultat}) \times \dots \times P(\text{Résultat})}{P(\text{Indices Multiples})}$$

Cette formule est souvent conceptualisée comme le produit des vraisemblances de chaque indice multiplié par la probabilité a priori du résultat, divisé par la probabilité de tous les indices. Ce calcul ajuste les probabilités basées sur la fréquence et l'indépendance des indices, offrant une prédiction finale (a posteriori):

- Si la probabilité d'un indice, étant donné un résultat, est de 1, cela ne change pas le produit.

- Si la probabilité d'un indice particulier, étant donné le résultat est de 0, cela annule toute la probabilité, permettant d'exclure certains résultats en présence de preuves contradictoires.
- La division par $P(\text{Indices})$ normalise le résultat mais est souvent omise dans les calculs pratiques, facilitant ainsi le processus.

Cette méthode attribue une haute probabilité aux résultats les plus communs et réduit celle des issues moins probables, en s'appuyant sur ce qu'on appelle les taux de base pour ajuster les prédictions.

L'approche de Naïve Bayes, malgré ses suppositions simplificatrices, s'avère très efficace, notamment dans la classification de textes où les dépendances entre mots sont souvent négligées pour la simplicité des calculs.

Dans le cadre du classificateur naïf de Bayes, la probabilité conditionnelle d'une caractéristique donnée d'une classe (c'est-à-dire $p(x_1 \mid y_i)$) est plus facile à estimer à partir des données. Pour ce faire, l'algorithme stocke les distributions de probabilité des caractéristiques pour chaque classe de manière indépendante. Par exemple, si l'on a 5 classes et 10 caractéristiques, il faudra stocker 50 distributions de probabilité différentes. Le type de distribution utilisé dépend de la nature des caractéristiques :

- **Caractéristiques binaires** (Oui/Non, Vrai/Faux, 0/1) : on utilise une distribution de Bernoulli, qui modélise la probabilité de succès ou d'échec pour un essai donné.
- **Caractéristiques discrètes** (comme les comptes de mots) : on utilise une distribution multinomiale, adaptée pour des variables discrètes avec plus de deux résultats possibles.
- **Caractéristiques continues** : on utilise une distribution gaussienne (normale), qui modélise des données continues souvent trouvées dans des phénomènes naturels.

Le classificateur naïf de Bayes est souvent nommé en fonction du type de distribution utilisé pour les caractéristiques (par exemple, "classificateur naïf de Bayes gaussien"). Pour des ensembles de données comprenant des types de caractéristiques mixtes, il peut être nécessaire d'utiliser différents types de distributions pour différentes caractéristiques.

En combinant toutes ces informations, l'algorithme naïf de Bayes peut facilement calculer la probabilité d'appartenance à une classe donnée en fonction des valeurs des caractéristiques ($p(y_i \mid x_1, x_2, \dots, x_n)$). Cela signifie que pour chaque classe y_i , l'algorithme :

1. Calcule la probabilité de chaque caractéristique donnée cette classe, en utilisant la distribution appropriée.
2. Multiplie ces probabilités individuelles entre elles pour obtenir une probabilité totale pour cette classe.

Ce processus est répété pour chaque classe, permettant à l'algorithme de comparer les probabilités et de déterminer la classe la plus probable pour une nouvelle observation.

Par exemple, si vous avez un ensemble de données où chaque observation est décrite par plusieurs caractéristiques (comme la couleur, la taille et le poids d'un fruit), l'algorithme va :

- Utiliser une distribution de Bernoulli pour la probabilité que le fruit soit de telle couleur donnée la classe.
- Utiliser une distribution gaussienne pour la probabilité que le fruit ait telle taille donnée la classe.
- Utiliser une distribution multinomiale pour la probabilité que le fruit ait tel nombre de graines donnée la classe.

En multipliant ces probabilités pour chaque classe et en les comparant, l'algorithme détermine la classe à laquelle le fruit a le plus de chances d'appartenir. Cela simplifie considérablement le processus de classification des nouvelles données, car il utilise des calculs de probabilité bien définis pour faire des prédictions basées sur les caractéristiques observées.

3.1.2 Exemple d'application

Exemple e-mail SPAM

Naïve Bayes est couramment utilisé pour classifier les e-mails en {SPAM, NON SPAM} en se basant sur la fréquence des mots. Cet exemple permet aussi de très bien cerner l'utilisation des algorithmes de Naïve Bayes.

Cet algorithme part de l'hypothèse que l'apparition des mots dans un e-mail est indépendante des autres mots, ce qui signifie que la présence d'un mot n'influence pas la présence d'un autre mot.

Pour chaque e-mail, l'algorithme Naïve Bayes calcule la probabilité que l'e-mail soit un SPAM ou un NON SPAM en se basant sur la fréquence des mots. Par exemple, des mots comme "gratuit", "offre" ou "gagner" apparaissent fréquemment dans les e-mails SPAM, tandis que des mots comme "réunion", "rapport" ou "projet" sont plus courants dans les e-mails NON SPAM. En analysant la fréquence de chaque mot dans les e-mails marqués comme SPAM et NON SPAM, l'algorithme peut estimer la probabilité qu'un nouvel e-mail appartienne à l'une ou l'autre catégorie.

L'hypothèse d'indépendance de Naïve Bayes, bien que simplificatrice, permet de calculer efficacement les probabilités. Cette hypothèse stipule que chaque mot dans un e-mail contribue de manière indépendante à la probabilité totale de l'e-mail d'être SPAM ou NON SPAM. D'un point de vue linguistique et sémantique, l'hypothèse d'indépendance est souvent incorrecte. En réalité, les mots dans un texte ne sont pas indépendants les uns des autres.

Par exemple, dans la phrase "gagner un prix gratuit", les mots "gagner" et "gratuit" sont sémantiquement liés et leur apparition conjointe augmente la probabilité que l'e-mail soit un SPAM.

Exemple avec le type de fruit

Considérant l'exemple des fruits, pour un fruit nouveau décrit comme long, sucré et jaune, Naïve Bayes calcule les probabilités pour déterminer s'il est une banane, une orange ou un autre fruit. La vraisemblance de chaque classe est ajustée par la probabilité a priori, et la classe avec la probabilité a posteriori la plus élevée est choisie.

En combinant ces informations, Naïve Bayes aide à déterminer objectivement la classification du fruit en se basant sur les données d'entraînement et les caractéristiques observées, même si les attributs sont traités comme indépendants.

Classification Multi-Classes avec Naive Bayes — Prédiction du Type de Fruit

Nous avons un ensemble de 1000 fruits divisés en trois catégories: Banane, Orange, et Autre. Chaque fruit est caractérisé par ses attributs: longueur, douceur (sucré ou non), et couleur jaune. Voici la répartition:

Type	Long	Petit ($\neg$ Long)	Sucré	$\neg$ Sucré	Jaune	Pas Jaune	Total
Banane	400	100	350	150	450	50	500
Orange	0	300	150	150	300	0	300
Autre fruit	100	100	150	50	50	150	200
Total	500	500	650	350	800	200	1000

Supposons que nous devons classifier un fruit inconnu qui est long, sucré et jaune. Les probabilités à calculer sont:

$$P(\text{Banane} \mid \text{long, jaune, sucré}) = \frac{P(\text{Long} \mid \text{Banane}) \times P(\text{Sucre} \mid \text{Banane}) \times P(\text{Jaune} \mid \text{Banane}) \times P(\text{Banane})}{P(\text{Long}) \times P(\text{Sucre}) \times P(\text{Jaune})}$$

Avec les données fournies:

$$P(\text{Banane}) = 0.5$$

$$P(\text{Long} \mid \text{Banane}) = 0.8$$

$$P(\text{Sucre} \mid \text{Banane}) = 0.7$$

$$P(\text{Jaune} \mid \text{Banane}) = 0.9$$

$$P(\text{Long}) = 0.5$$

$$P(\text{Sucre}) = 0.65$$

$$P(\text{Jaune}) = 0.8$$

Calculons la probabilité que ce soit une banane:

$$P(\text{Banane} \mid \text{long, jaune, sucre}) = \frac{0.8 \times 0.7 \times 0.9 \times 0.5}{0.5 \times 0.65 \times 0.8} \approx 0.969$$

Avec cette même logique on obtient :

$$P(\text{Orange} \mid \text{long, jaune, sucre}) = 0$$

$$P(\text{Autre Fruit} \mid \text{long, jaune, sucre}) \approx 0.072$$

On remarque que la probabilité que notre fruit soit une banane est largement plus grande que celle des autres probabilités.

On classifie donc notre fruit inconnu comme étant une banane.

3.2 Implémentation avec Python

3.2.1 Présentation du jeu de données

Pour ce tutoriel, on utilisera le jeu de données IRIS. Le dataset IRIS a été introduit pour la première fois par le statisticien et biologiste britannique Ronald A. Fisher dans son article de 1936 intitulé *The use of multiple measurements in taxonomic problems*. Depuis, il est largement utilisé comme un exemple de classification statistique et est souvent employé pour tester de nouveaux algorithmes en apprentissage automatique. Ce dernier est une base de données regroupant les caractéristiques de trois espèces de fleurs d'Iris, à savoir Setosa, Versicolour et Virginica. Chaque ligne de ce jeu de données est une observation des caractéristiques d'une fleur d'Iris. Ce dataset décrit les espèces d'Iris par quatre propriétés : longueur et largeur de sépales ainsi que longueur et largeur de pétales. La base de données comporte 150 observations (50 observations par espèce).

L'objectif est généralement de prédire la sous-espèce de la plante en se basant sur ces mesures. C'est l'un des jeux de données les plus connus en apprentissage automatique, et il est souvent inclus comme un dataset "d'entraînement" avec des frameworks pour apprendre les étapes générales de l'apprentissage automatique. Par conséquent, il servira d'exemple pour illustrer les tâches associées à l'apprentissage automatique.

Ce jeu de données est particulièrement apprécié pour les démonstrations en raison de sa simplicité et de sa taille réduite, ce qui permet de visualiser facilement les relations entre les différentes variables.

Code pour afficher un DataFrame pandas avec le dataset IRIS

```
import pandas as pd
from sklearn import datasets

# Chargement du dataset IRIS
iris = datasets.load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

# Creation d'un DataFrame pandas
df = pd.DataFrame(X, columns=feature_names)
df['species'] = pd.Categorical.from_codes(y, target_names)

# Affichage des premieres lignes du DataFrame
print(df.head(150))
```

Voici l'output du code ci-dessus, affichant les premières lignes du DataFrame créé à partir du dataset IRIS :

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
..	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

	species
0	setosa
1	setosa
2	setosa
3	setosa
4	setosa
..	...
145	virginica
146	virginica
147	virginica
148	virginica
149	virginica

[150 rows x 5 columns]

Les codes que vous trouverez ci-dessous sont visibles sur un repository Github via ce lien : <https://github.com/tfallouh>

3.2.2 Programme Manuel

Code classification Naïve Bayésienne sur IRIS "façon manuelle"

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split

# Chargement du dataset IRIS
iris = datasets.load_iris()
X, y = iris.data, iris.target
classes = np.unique(y)

# Separation du dataset en donnees d'entrainement et de test en utilisant
la stratification
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)

# Affichage des formes des donnees pour verifier la division
print(f"X_train-shape: {X_train.shape}")
print(f"X_test-shape: {X_test.shape}")
print(f"y_train-shape: {y_train.shape}")
print(f"y_test-shape: {y_test.shape}")

# Calculs des probabilites a priori pour chaque classe
def calculate_prior(y_train, classes):
    priors = {}
    total_count = len(y_train)
    class_counts = np.bincount(y_train)
    # Compte le nombre d'instances pour chaque classe
    for c in classes:
        priors[c] = class_counts[c] / total_count
        # Calcule la probabillite a priori pour chaque classe
    return priors

# Affichage des probabilites a priori
priors = calculate_prior(y_train, classes)
print("Probabilites a priori pour chaque classe:")
for cls, prob in priors.items():
    print(f"Classe {cls}: {prob:.4f}")

# Calculs des moyennes et variances pour chaque attribut par classe
def calculate_mean_variance(X_train, y_train, classes):
    mean_var = {}
    for c in classes:
        X_c = X_train[y_train == c]
        # Selectionne les instances de la classe c
        mean_var[c] = {
            'mean': np.mean(X_c, axis=0),
            # Calcule la moyenne des attributs pour la classe c
            'var': np.var(X_c, axis=0),
            # Calcule la variance des attributs pour la classe c
        }
    return mean_var
```

```

# Affichage des moyennes et variances des attributs par classe
mean_var = calculate_mean_variance(X_train, y_train, classes)
print("\nMoyennes et variances pour chaque attribut par classe:")
for cls, stats in mean_var.items():
    print(f"Classe-{cls}:")
    print(f"--Moyennes: {stats['mean']}")
    print(f"--Variances: {stats['var']}")

# Densite de probabilite d'une distribution normale
def gaussian_density(x, mean, var):
    exponent = np.exp(-((x - mean) ** 2) / (2 * var))
    # Calcule la partie exponentielle de la densite
    return (1 / np.sqrt(2 * np.pi * var)) * exponent
    # Calcule la densite de probabilite

# Calcul de la probabilite posterieure pour une instance donnee
def calculate_posterior(X, priors, mean_var, classes):
    posteriors = []
    for c in classes:
        prior = np.log(priors[c])
        # Prend le logarithme de la probabilite a priori
        conditional = np.sum
        (np.log(gaussian_density(X, mean_var[c]['mean'], mean_var[c]['var'])))
        # Calcule la somme des logs des vraisemblances
        posterior = prior + conditional
        # Calcule la log-probabilite posterieure
        posteriors.append(posterior)
    return classes[np.argmax(posteriors)], posteriors
    # Retourne la classe avec la plus grande probabilite posterieure
    et les probabilites

# Entrainement du modele
priors = calculate_prior(y_train, classes)
mean_var = calculate_mean_variance(X_train, y_train, classes)

# Prediction de la classe pour une nouvelle instance
def predict(X, priors, mean_var, classes):
    predictions = []
    for instance in X:
        predicted_class, posteriors = calculate_posterior(instance, priors, mean_var, classes)
        predictions.append(predicted_class)
    return np.array(predictions)

# Predictions sur le dataset de test
y_pred = predict(X_test, priors, mean_var, classes)

# Evaluation de la precision
accuracy = np.mean(y_pred == y_test)
print(f"\nAccuracy: {accuracy * 100:.2f}%")

# Saisie de la nouvelle instance
print("\nEntrez les valeurs pour une nouvelle instance:")
new_instance = [
    float(input("Longueur du sepale (4.3 -- 7.9 cm) :")),
    float(input("Largeur du sepale (2.0 -- 4.4 cm) :")),
    float(input("Longueur du petale (1.0 -- 6.9 cm) :")),
    float(input("Largeur du petale (0.1 -- 2.5 cm) :"))
]

```

```

]

# Predire la classe pour la nouvelle instance
predicted_class, posteriors = calculate_posterior
(new_instance, priors, mean_var, classes)
print(f'\nPredicted class for the new instance: {predicted_class}')

# Affichage des log-probabilites posterieures pour la nouvelle instance
print("\nLog-probabilites posterieures pour la nouvelle instance:")
for cls, posterior in zip(classes, posteriors):
    print(f"Classe-{cls}: {posterior:.4f}")

# Affichage de la densite de probabillite pour la nouvelle instance
print("\nDensites-de-probabilite-pour-la-nouvelle-instance:")
for cls, stats in mean_var.items():
    densities = gaussian_density(new_instance, stats['mean'], stats['var'])
    print(f"Classe-{cls}: {densities}")

```

3.2.3 Construction du modèle

Le programme commence par l'importation des bibliothèques nécessaires, dont **numpy** pour les opérations mathématiques et **scikit-learn** pour la manipulation des données et l'apprentissage automatique. Ensuite, le jeu de données IRIS est chargé en utilisant la fonction **datasets.load_iris()** de **scikit-learn**. Cette fonction permet de charger facilement un jeu de données standard largement utilisé dans la recherche et les tutoriels d'apprentissage automatique.

Une fois les données chargées, elles sont divisées en deux parties : un ensemble d'entraînement et un ensemble de test. Pour cela, la fonction **train_test_split** est utilisée. Elle permet de diviser les données en respectant la proportion des classes dans chaque ensemble, grâce à l'argument **stratify**. Cette stratification est importante pour s'assurer que les classes (Setosa, Versicolor, Virginica) sont présentes dans les mêmes proportions dans les deux ensembles. Cette division permet d'entraîner le modèle sur une partie des données (70 %) tout en réservant l'autre partie (30 %) pour évaluer la capacité du modèle à généraliser sur de nouvelles données. La stratification est utilisée pour s'assurer que les proportions des différentes classes sont conservées dans les deux ensembles, ce qui est crucial pour éviter les biais.

La fonction **calculate_prior** est utilisée pour calculer les probabilités a priori pour chaque classe. Elle prend les étiquettes de l'ensemble d'entraînement et les classes uniques en entrée, compte les instances de chaque classe, et divise par le nombre total d'instances pour obtenir les probabilités a priori. Ces probabilités a priori sont essentielles car elles représentent la probabilité de chaque classe avant d'observer les données. Elles servent de base pour les calculs ultérieurs des probabilités postérieures.

Ensuite, la fonction **calculate_mean_variance** calcule les moyennes et les variances des attributs pour chaque classe (longueur et largeur des sépales, longueur et largeur des pétales). Pour chaque classe, la fonction filtre les instances correspondantes et calcule la moyenne et la variance des attributs. Ces statistiques descriptives sont nécessaires pour modéliser la distribution des attributs sous l'hypothèse de normalité, une hypothèse clé dans le classifieur naïf bayésien gaussien. Cette hypothèse est raisonnable ici, car les tailles des sépales et pétales sont des variables continues qui appartiennent à $\mathbb{R}$, ce qui rend l'utilisation de la distribution normale appropriée.

La fonction **gaussian_density** est chargée de calculer la densité de probabilité d'une distribution normale pour chaque attribut. Elle utilise la formule de la densité de probabilité gaussienne, qui dépend de la moyenne et de la variance des attributs pour une classe donnée. Cette densité est ensuite utilisée pour évaluer la vraisemblance qu'une instance appartienne à une certaine classe donnée ses caractéristiques (mesures des sépales et pétales). La fonction de densité gaussienne est choisie ici car elle permet de modéliser efficacement la probabilité des attributs continus (comme les tailles des sépales et pétales) en fonction de leur distribution observée dans les données.

Pour calculer les probabilités postérieures, la fonction **calculate_posterior** est utilisée. Elle prend en compte les probabilités a priori et les densités de probabilité des attributs. En prenant le logarithme de la probabilité a priori et en calculant la somme des logarithmes des densités de probabilité pour

chaque classe, elle obtient les log-probabilités postérieures. L'utilisation des logarithmes est une pratique courante pour éviter les problèmes de sous-dépassement numérique lors de la multiplication de nombreuses petites probabilités. La classe avec la plus grande log-probabilité postérieure est choisie comme prédiction, suivant le principe du maximum de vraisemblance.

La fonction **predict** utilise **calculate_posterior** pour prédire les classes des instances dans l'ensemble de test. Pour chaque instance, elle détermine la classe avec la plus grande log-probabilité postérieure et retourne un tableau des prédictions. Cela permet d'évaluer la performance du modèle sur des données qu'il n'a pas vues pendant l'entraînement, fournissant ainsi une estimation de sa capacité de généralisation.

Le modèle est ensuite évalué en calculant la précision des prédictions sur l'ensemble de test. La précision est mesurée en comparant les prédictions aux étiquettes réelles et en calculant la proportion des prédictions correctes. Cela donne une mesure simple mais efficace de la performance globale du modèle sur ce dataset (IRIS).

Enfin, le code permet de saisir une nouvelle instance et de prédire sa classe en utilisant les probabilités calculées. Les log-probabilités postérieures et les densités de probabilité pour la nouvelle instance sont affichées, fournissant une vue détaillée de la manière dont la prédiction a été faite. Cette partie interactive permet de tester le modèle en temps réel avec des données nouvelles et non vues.

3.2.4 Programme avec Scikit-Learn

Code classification Naïve Bayésienne sur IRIS "façon Automatique avec Scikit-Learn"

```
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# Chargement du dataset IRIS
iris = datasets.load_iris()
X = iris.data
y = iris.target
classes = iris.target_names

# Separation du dataset en donnees d'entrainement et de test en
utilisant la stratification
X_train, X_test, y_train, y_test = train_test_split
(X, y, test_size=0.3, random_state=42, stratify=y)

# Afficher les formes des donnees pour verifier la division
print(f"X_train-shape: {X_train.shape}")
print(f"X_test-shape: {X_test.shape}")
print(f"y_train-shape: {y_train.shape}")
print(f"y_test-shape: {y_test.shape}")

# Initialisation du classifieur bayesien naif
gnb = GaussianNB()

# Entrainement du modele
gnb.fit(X_train, y_train)

# Predictions sur le dataset de test
y_pred = gnb.predict(X_test)

# Evaluation de la precision
accuracy = accuracy_score(y_test, y_pred)
print(f'\nAccuracy: {accuracy*100:.2f}%')

# Affichage du rapport de classification
```

```

print("\\nClassification Report:")
print(classification_report(y_test , y_pred , target_names=classes))

# Affichage des probabilités a priori
print("\\nProbabilités a priori pour chaque classe:")
priors = gnb.class_prior_
for cls , prob in zip(classes , priors):
    print(f"Classe-{cls}:-{prob:.4f}")

# Affichage des moyennes et variances des attributs par classe
print("\\nMoyennes et variances pour chaque attribut par classe:")
means = gnb.theta_
variances = gnb.var_
for i , cls in enumerate(classes):
    print(f"Classe-{cls}:")
    print(f"  Moyennes:-{means[i]}")
    print(f"  Variances:-{variances[i]}")

# Saisie une nouvelle instance
print("\\nEntrez les valeurs pour une nouvelle instance:")
new_instance = np.array([
    float(input("Longueur du sepale (4.3 -- 7.9 cm):-")),
    float(input("Largeur du sepale (2.0 -- 4.4 cm):-")),
    float(input("Longueur du petale (1.0 -- 6.9 cm):-")),
    float(input("Largeur du petale (0.1 -- 2.5 cm):-"))
]).reshape(1 , -1)

# Afficher les valeurs de la nouvelle instance
print("\\nValeurs de la nouvelle instance:")
print(f"Longueur du sepale:-{new_instance[0 , -0]} cm")
print(f"Largeur du sepale:-{new_instance[0 , -1]} cm")
print(f"Longueur du petale:-{new_instance[0 , -2]} cm")
print(f"Largeur du petale:-{new_instance[0 , -3]} cm")

# Prediction de la classe pour la nouvelle instance
predicted_class = gnb.predict(new_instance)[0]
predicted_class_name = classes[predicted_class]
print(f'\\nPredicted class for the new instance:-{predicted_class_name}')

# Affichage des log-probabilités postérieures pour la nouvelle instance
log_posteriors = gnb._joint_log_likelihood(new_instance)[0]
print("\\nLog-probabilités postérieures pour la nouvelle instance:")
for cls , log_posterior in zip(classes , log_posteriors):
    print(f"Classe-{cls}:-{log_posterior:.4f}")

# Affichage des densités de probabilité pour la nouvelle instance
def gaussian_density(x, mean, var):
    exponent = np.exp(-((x - mean) ** 2) / (2 * var))
    return (1 / np.sqrt(2 * np.pi * var)) * exponent

print("\\nDensités de probabilité pour la nouvelle instance:")
for i , cls in enumerate(classes):
    densities = gaussian_density(new_instance , means[i] , variances[i])
    print(f"Classe-{cls}:-{densities}")

```

Le code utilise **train_test_split** pour diviser les données en ensembles d'entraînement et de test, tout en maintenant les proportions de classes grâce à la stratification. **GaussianNB()** initialise un classifieur naïf bayésien gaussien, qui est ensuite entraîné via la méthode **fit**. Les prédictions sont effectuées avec

predict, et la précision est évaluée avec **accuracy_score**. Le rapport de classification, généré par **classification_report**, fournit des métriques telles que la précision et le F1-score.

Les probabilités a priori et les moyennes/variances des attributs pour chaque classe sont calculées automatiquement et accessibles via les attributs **class_prior_**, **theta_** et **var_**. Le code permet également de prédire la classe d'une nouvelle instance, en affichant les log-probabilités postérieures avec **_joint_log_likelihood** et en utilisant une fonction de densité gaussienne pour calculer la vraisemblance des attributs.

Cette version fait appel aux fonctions intégrées de **scikit-learn**, simplifiant ainsi l'implémentation tout en automatisant les calculs.

3.2.5 Output du programme

Le programme commence par diviser le jeu de données IRIS en ensembles d'entraînement et de test. Après la division, les formes des ensembles sont affichées :

```
X_train shape: (105, 4)
X_test shape: (45, 4)
y_train shape: (105,)
y_test shape: (45,)
```

Cela signifie que 105 observations avec 4 attributs chacune (longueur et largeur des sépales et pétales) sont utilisées pour entraîner le modèle, tandis que 45 observations sont réservées pour le test. Les étiquettes associées des classes (Setosa, Versicolor, Virginica) sont également divisées de manière proportionnelle entre les ensembles d'entraînement et de test.

Les probabilités a priori pour chaque classe sont affichées et sont toutes égales à 0.3333 :

```
Probabilités a priori pour chaque classe :
Classe 0: 0.3333
Classe 1: 0.3333
Classe 2: 0.3333
```

Cela reflète la répartition équilibrée des classes dans le dataset IRIS, où chaque classe possède exactement 50 échantillons sur un total de 150. Ainsi, la probabilité a priori pour chaque classe est de $\frac{50}{150} = 0.3333$.

Les moyennes et variances pour chaque attribut sont calculées pour chaque classe. Ces statistiques sont utilisées pour modéliser la distribution des attributs sous l'hypothèse d'une distribution gaussienne. Par exemple, pour la classe 0 (Setosa), la longueur moyenne du sépale est d'environ 4.99 cm avec une variance de 0.1033 :

```
Classe 0:
Moyennes: [4.98857143 3.42571429 1.48571429 0.24]
Variances: [0.10329796 0.1739102 0.02293878 0.00925714]
```

Des moyennes et variances similaires sont calculées pour les autres classes (Versicolor et Virginica).

Le modèle atteint une précision (accuracy) de 91.11%, calculée sur l'ensemble de test :

```
Accuracy: 91.11%
```

L'accuracy, ou précision, est une mesure couramment utilisée pour évaluer la performance globale d'un modèle de classification. Elle est définie comme la proportion des prédictions correctes par rapport à l'ensemble des prédictions effectuées. Plus précisément, elle est calculée comme suit :

$$\text{Accuracy} = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total de prédictions}}$$

Dans ce cas, une précision de 91.11% signifie que sur les 45 observations de l'ensemble de test, environ 41 d'entre elles ont été correctement classées par le modèle. Cela montre que le modèle a bien appris à distinguer les différentes classes de fleurs sur la base des attributs fournis.

Une nouvelle instance est ensuite entrée par l'utilisateur, avec les caractéristiques suivantes :

```
Longueur du sépale: 5.0 cm
Largeur du sépale: 4.0 cm
Longueur du pétale: 3.0 cm
```

Largeur du pétale: 2.0 cm

Le modèle prédit que cette fleur appartient à la classe 2, qui correspond à l'espèce Virginica :

Predicted class for the new instance: 2

Les log-probabilités postérieures pour chaque classe sont également affichées. Ce sont les probabilités logarithmiques que la nouvelle instance appartienne à chaque classe. La classe avec la plus grande log-probabilité postérieure est choisie comme prédiction. Ici, la classe 2 a la plus grande log-probabilité postérieure :

Log-probabilités postérieures pour la nouvelle instance :

Classe 0: -216.7757

Classe 1: -21.9850

Classe 2: -19.6198

Enfin, les densités de probabilité gaussiennes sont calculées pour chaque attribut de la nouvelle instance en fonction de la distribution des attributs dans chaque classe. Ces densités montrent à quel point les attributs de la nouvelle instance sont proches des moyennes des attributs dans chaque classe. Par exemple, pour la classe 2 (Virginica), la densité pour la largeur du pétale est la plus élevée, ce qui explique pourquoi cette classe a été prédite :

Densités de probabilité pour la nouvelle instance :

Classe 0: [[1.24047961e+00 3.70636805e-01 5.17164273e-22 9.04589616e-73]]

Classe 1: [[1.25497795e-01 1.12611894e-04 2.32660331e-02 2.58344340e-03]]

Classe 2: [[2.18427455e-02 1.76766469e-02 1.53951632e-05 1.52151903e+00]]

Conclusion

La classification naïve bayésienne est un algorithme de classification probabiliste qui repose sur l'hypothèse d'indépendance conditionnelle des attributs. Cela signifie que, pour une classe donnée, chaque attribut est supposé être indépendant des autres. Bien que cette hypothèse soit souvent irréaliste dans les scénarios du monde réel, la classification naïve bayésienne s'avère étonnamment performante dans de nombreux cas, notamment grâce à sa simplicité et sa rapidité d'exécution. Par exemple, pour des jeux de données comme IRIS, où les attributs (longueur et largeur des sépales et pétales) peuvent être approximatés par des distributions gaussiennes, l'algorithme offre des résultats convaincants.

Cependant, cette simplicité peut devenir une faiblesse. Dans les données réelles, les attributs ne sont souvent pas indépendants. Par exemple, dans un jeu de données médical, le taux de cholestérol ou la tension artérielle d'une personne peut-être lié à son âge. Ignorer ces liens peut entraîner des erreurs de prédiction, car l'algorithme ne prend pas en compte les relations complexes entre les attributs.

Un autre inconvénient est que la version gaussienne de l'algorithme suppose que les attributs suivent une distribution normale (en forme de cloche). Si ce n'est pas le cas, comme dans des jeux de données où les valeurs sont très asymétriques ou ont plusieurs pics, l'algorithme peut faire des erreurs. Par exemple, dans des données financières ou économiques, où les valeurs peuvent varier énormément, cette approximation n'est souvent pas adaptée.

L'algorithme naïf bayésien peut aussi avoir du mal avec les données contenant des valeurs manquantes ou des valeurs extrêmes (outliers). Comme il repose sur des calculs précis des moyennes et des variances, ces valeurs peuvent fausser les résultats si elles ne sont pas traitées correctement. On retrouve par exemple le "problème de fréquence zéro", qui survient lorsque certaines catégories observées dans les données de test ne sont pas présentes dans les données d'entraînement. Dans ce cas, l'algorithme attribue une probabilité de zéro à cette catégorie, empêchant toute prédiction. Pour y remédier, des techniques de lissage comme la correction de Laplace sont souvent utilisées.

Enfin, l'algorithme naïf bayésien peut avoir des difficultés à traiter des ensembles de données déséquilibrés, où certaines classes sont beaucoup plus fréquentes que d'autres. Il a tendance à favoriser les classes majoritaires, ce qui peut poser problème dans des domaines comme la détection de fraude ou le diagnostic médical, où les cas rares sont souvent les plus importants à détecter.

Malgré ces limitations, la classification naïve bayésienne reste utile pour des problèmes simples ou pour des jeux de données bien équilibrés. En résumé, bien que limité par certaines hypothèses et simplifications, cet algorithme est rapide, facile à comprendre et performant dans de nombreuses applications courantes.

References

Bibliographie

- Vose, David. *Risk Analysis: A Quantitative Guide*. John Wiley & Sons, 2008.
- Gelman, Andrew, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, et Donald B. Rubin. *Bayesian Data Analysis*. 3rd ed., Chapman and Hall/CRC, 2013.
- Bishop, Christopher M. *Pattern Recognition and Machine Learning*. Springer, 2006.

Sitographie

- Analyse approfondie de la fonction de masse de probabilité (PMF) et des distributions discrètes.
- Explication des distributions conjointes des probabilités sur Bibmath.net.
- Documentation sur l'utilisation du classifieur Naive Bayes dans scikit-learn.
- Tutoriel pour atteindre 100% d'accuracy en utilisant Naive Bayes sur le dataset Iris sur Kaggle.
- Tutoriel sur la classification des données Iris en utilisant Naive Bayes sur Kaggle.
- Explication complète de la classification naïve bayésienne sur Wikipédia.
- Introduction au concept de maximum de vraisemblance sur Wikipédia.
- Tutoriel détaillé sur les classifieurs Naive Bayes sur GeeksforGeeks.
- Diapositives du cours sur le Naive Bayes à Stanford University.
- Cours sur le classifieur bayésien de l'Université de Montréal.
- Chapitre sur les probabilités conditionnelles par l'Université de Toulouse.
- Document sur les probabilités édité par l'Université Lyon 1.

Vidéos

- How to implement Naive Bayes from scratch with Python.
- Naive Bayes, Clearly Explained!!!.
- Classification Naïve Bayésienne - Naive Bayes classifier.